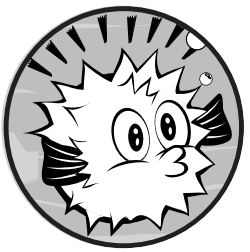
8

DISKS AND FILESYSTEMS

Oh, my head hurts bad.
Rings of ones and zeros, ouch!
Filesystems hide them.



Proper data management is perhaps a systems administrator's most vital duty. You can replace almost every computer component, but the data on your disk is irreplaceable. Perhaps that data isn't important or it's backed up, but losing files will ruin your day. As a sysadmin, you must protect important data by carefully managing your disks and filesystems.

We covered the basics of disklabels and MBR partitions in Chapter 2, but OpenBSD lets you use and abuse disks and filesystems in any number of ways. You'll learn how in this chapter.

Device Nodes

A *device node* is a file that provides a logical interface to a piece of hardware. By reading from a device node, sending data to it, or using a command on it, you're telling the operating system to perform an action on a piece of hardware or, in some cases, a logical device.

Different devices behave differently when data is sent to them. For example, writing to the console makes text appear on the screen or terminal, while writing to a disk device puts data on that disk. (OpenBSD puts device nodes in */dev* and disallows device nodes on other filesystems.)

Many disk management programs expect to be given a device name as an argument. Unfortunately, device node names are frequently cryptic and vary widely among operating systems—even on closely related operating systems that run on the same hardware. To simplify your life just a bit, Table 8-1 lists the device node names for common OpenBSD disk devices.

Table 8-1: Common Disk Device Node Names

Device Node	Description
*/dev/fd**	Floppy disk (block)
*/dev/rfd**	Floppy disk (raw)
*/dev/wd**	IDE and some SATA disks (block)
*/dev/rwd**	IDE and some SATA disks (raw)
*/dev/sd**	SCSI/SAS/SATA/USB/RAID/non-IDE disk (block)
*/dev/rsd**	SCSI/SAS/SATA/USB/RAID/non-IDE disk (raw)
*/dev/cd**	CD/DVD drive (block)

Device names also have a number that tells you which instance of that device it refers to. The numbering starts at 0. The first IDE hard drive is */dev/wd0*, */dev/wd1* is the second, and */dev/cd1* is the second CD drive.

Every partition is assigned a letter. For example, the root partition is *a*, the swap area is *b*, the whole disk is *c*, and so on. Each partition also has a separate device node, the result of appending the partition letter to your disk device name. For example, if you install to a single IDE drive, your root partition is */dev/wd0a*.

Raw and Block Devices

Notice in Table 8-1 that devices are listed in either block or raw (character) mode. This refers to how the devices are accessed.

Block Devices

Hard disks are usually accessed using a block device node (sometimes called a *cooked* device node). When accessing a device as a block, data transmitted to or from the device is *buffered*, or collected until there is sufficient data to

make accessing the device worth the trouble. Block devices are generally considered more efficient than raw devices.

The device nodes for block devices are named after the device driver; for example, */dev/wd3*.

Raw Devices

Raw devices are sometimes called *character* devices, because they access a device one character at a time. If you need to control exactly how data appears on a disk (for example, when creating a filesystem) use a raw device. Raw device nodes have an *r* in front of their name, as in */dev/rwd3*.

Raw devices do no buffering. When you tell your system to write to a raw device, the data is transmitted immediately. Raw mode works best with software that provides its own buffering or that wants to arrange data in a specific way.

Here's an easy way to remember the difference between block and raw throughput: Say you spill a bottle of aspirin. If you pick up each aspirin individually and deposit it directly in the bottle, you're doing an unbuffered, or raw, transfer. If you pick up the aspirin with your right hand and collect them in your left, then dump a bunch into the bottle at once (along with all of the dirt from your floor), you're doing a buffered transfer.[1]

Choosing Your Mode

Address disks (and many other devices) as raw or block by choosing the corresponding device node. Some programs expect to access raw devices, while others expect block devices. If a program opens */dev/sd1a*, it's accessing partition *a* on disk *sd1* as a block device. If it opens */dev/rsd1a*, it's accessing the exact same partition as a character device.

Regardless of the mode, the underlying hardware remains the same; the only thing that changes is how you exchange information with the device.

Device Attachment vs. Device Name

Not long ago, most disks were permanently affixed to a single physical location on the system. If your computer had two IDE buses, each with two hard drives, the operating system knew exactly where to find them, usually at */wd1* and */wd2*. A SCSI disk had a SCSI ID and a logical unit number (LUN), and changing them required rebooting the computer. Traditionally, you could use the disk's location in the system to identify the disk. For example, a booting i386 computer would find the root partition by looking for the hard drive attached to the first port on the first IDE controller, finding the *a* partition on that disk, and reading the filesystem table from that disk. You could go into the BIOS to tell the computer to look for the root partition on a different disk, but the computer still identified the disk by where it was physically attached to the computer.

---

1. If it's buffered aspirin, then you're doing buffered buffered aspirin transfers. But let's not go there.

Today, disks can appear and disappear from multiple locations on the system. For example, you might attach and remove several flash drives as needed, or hot swap Serial Attached SCSI (SAS) or Serial ATA (SATA) drives from bus to bus. Physical location is no longer a safe way to identify a disk. While */dev/sd0* is the device node for the first SCSI disk, you cannot assume that the disk currently attached to the first SCSI port is the same disk that was plugged in there the last time the system booted. OpenBSD labels actual disks with unique IDs, as discussed in the next section.

DUIDs and /etc/fstab

All OpenBSD platforms use the disklabel to identify partitions and other information about a disk. When you label a disk (as we did in Chapter 3 and will do by hand later this chapter), `disklabel` adds a *disklabel unique identifier*, or DUID, to the disk label. The DUID is a unique hexadecimal number that lets OpenBSD identify a specific disk.

To find a disk's DUID, pass the device name to `disklabel` and look for the `duid` entry:

```
# disklabel sd0
...
duid: 55128c3700af5491
...
```

The disk currently attached as `sd0` has a DUID of `55128c3700af5491`. Even if you physically move the disk so that it becomes `sd9` or `sd18`, OpenBSD can use the DUID to uniquely identify this disk.

OpenBSD uses the filesystem table */etc/fstab* to map filesystems on a disk to mount points using either the disk location or the DUID. Each filesystem appears on its own line in */etc/fstab*, as shown here:

```
❶ 55128c3700af5491.b ❷none ❸swap ❹sw
55128c3700af5491.a / ffs rw 1 1
55128c3700af5491.k /home ffs rw,nodev,nosuid 1 2
55128c3700af5491.d /tmp ffs rw,nodev,nosuid 1 2
...
```

We'll focus on the first entry to explore what's going on here. The first field, `55128c3700af5491.b` ❶, is the location of the partition. Whereas older systems used the disk device name and the partition letter (such as */dev/sd0a*), newer systems can use the DUID, a period, and the partition letter (as in `55128c3700af5491.a`). By using DUIDs in the filesystem table, OpenBSD can always mount the same disk at the same location, no matter how it's attached.

The second field, `none` ❷, lists the *mount point*, which is the directory where the filesystem is attached to the directory tree. Every partition you can write files to is attached to a mount point (such as */usr*, */var*, and so on), with one partition being the root partition (/). Swap space uses a mount point of `none`.

Next, `swap` ❸, is the filesystem type. The standard OpenBSD partition uses type `ffs`, the UNIX Fast File System. Other options include, but are not limited to, `msdos` (Microsoft-style FAT partitions), `mfs` (Memory File System), and `cd9660` (CD).

The fourth field, `sw` ❹, shows the mount options used for this filesystem. I'll cover mount options in more detail in "FFS Mount Options" on page 135, but here are a few that frequently appear in */etc/fstab*:

`ro` The filesystem is mounted read-only. Not even root can write to it.

`rw` The filesystem is mounted read-write.

`nodev` Device nodes are not interpreted.

`nosuid` `setuid` files are forbidden.

`noauto` OpenBSD won't automatically mount the filesystem at boot or when running `mount -a`. This option is useful for removable media drives that might not have media in them, such as CD and USB flash drives.

The fifth field indicates whether `dump(8)` should back up this filesystem. If this field is 0 (or absent), `dump` doesn't routinely back up the filesystem. Otherwise, the number given is the minimum dump level needed to back up the filesystem.

The last field is the *pass number*. It tells `fsck` when to check the filesystem during boot. Filesystems with a pass number of 1 are checked first, filesystems with a pass number of 2 are checked second, and so on. A pass number of 0 tells `fsck` to not check the filesystem during boot. If a filesystem doesn't have a pass number, it's equivalent to 0.

I strongly recommend using DUIDs in */etc/fstab* and anywhere else, rather than using device node names. While a device node name might change, a DUID will not.

MBR Partitions and fdisk(8)

Some hardware platforms have specific ideas about disk partitioning that differ from what OpenBSD expects. For example, the i386 and amd64 platforms expect to find MBR partitions on hard drives, and OpenBSD accommodates this quirk by putting its own disklabel partitions inside MBR partitions. We briefly touched on creating partitions during the installation process, but if you add hard drives to an existing system, you'll need to edit the MBR partition table by hand using `fdisk(8)`.

My particular test system has two hard drives: *wd0* and *wd1*. I think that *wd1* is completely blank but before I can use this drive, I need to verify that it is empty, and then create MBR partitions. While `fdisk` has all sorts of commands to edit disks, I find the simplest way is to use the interactive disk editor. Run `fdisk -e` and give it the device node for the new disk.

```
# fdisk -e wd1
Enter 'help' for information
fdisk: 1>
```

The editor is minimal, but lets you view, add, remove, and edit MBR partitions. If you forget the commands at any time, entering `help` will print out all the commands `fdisk` supports.

Viewing MBR Partitions

To see the MBR partitions on the current disk, enter **`print`** or **`p`**. Here's an example:

```
fdisk: 1> print
Disk: wd1       geometry: 2088/255/63 [33554304 Sectors]
Offset: 0       Signature: 0x0
            Starting         Ending         LBA Info:
 #: id      C   H   S -      C   H   S [       start:        size ]
--------------------------------------------------------------------------------
 0: 00      0   0   0 -      0   0   0 [           0:           0 ] unused
 1: 00      0   0   0 -      0   0   0 [           0:           0 ] unused
 2: 00      0   0   0 -      0   0   0 [           0:           0 ] unused
 3: 00      0   0   0 -      0   0   0 [           0:           0 ] unused
```

The first line shows the disk geometry (as discussed in Chapter 2). Every value in this disk's MBR table is set to 0, meaning that it has no configured partitions.

Adding and Removing Partitions

Say we want to create an MBR partition on this disk. I habitually use partition 0, but the OpenBSD installer usually uses partition 3. The specific number you pick doesn't matter unless you want multiple MBR partitions on the disk.

To edit a partition, enter **`edit`** or **`e`** followed by the partition number. For example, to edit partition 0, enter the following:

```
fdisk: 1> e 0
            Starting         Ending         LBA Info:
 #: id      C   H   S -      C   H   S [       start:        size ]
--------------------------------------------------------------------------------
 0: 00      0   0   0 -      0   0   0 [           0:           0 ] unused
❶ Partition id ('0' to disable)  [0 - FF]: [0] (? for help) a6
  Do you wish to edit in CHS mode? [n]
❷ offset: [0]
❸ size: [0] *
```

WARNING *Conveniently, `fdisk` prints the current information on this MBR partition. Make sure it's the partition you think it is before you muck it up.*

First, at ❶, set a partition ID. This is a label indicating what kind of filesystem will be on the disk. OpenBSD uses partition ID `a6`, so enter that.

The offset at ❷ is the number of sectors from the beginning of the disk to the start of the partition. We want to use this entire disk for OpenBSD, so set it to `0`.

Finally, the size at ❸ is the number of sectors the MBR partition fills. There is no need to copy the number of sectors in the disk here; OpenBSD `fdisk` uses * to mean "all free space."

Now print the MBR table again to check your work.

```
fdisk:*1> p
Disk: wd1       geometry: 2088/255/63 [33554304 Sectors]
Offset: 0       Signature: 0x0
            Starting         Ending         LBA Info:
 #: id      C   H   S -      C   H   S [       start:        size ]
-------------------------------------------------------------------------------
 0: A6      0   0   1 -   2088 167  63 [           0:    33554304 ] OpenBSD
 1: 00      0   0   0 -      0   0   0 [           0:           0 ] unused
 2: 00      0   0   0 -      0   0   0 [           0:           0 ] unused
 3: 00      0   0   0 -      0   0   0 [           0:           0 ] unused
```

Notice that the entry for partition 0 is type A6 and extends from cylinder 0, head 0, sector 1, to cylinder 2088, head 167, sector 63. It fills 33,554,304 sectors—the same as the number of sectors in the disk. This MBR partition fills the entire disk.

If you had recycled this disk from another operating system, it would probably have a partition already on it. To remove a partition, edit the partition and set its partition ID to 0.

Making a Partition Bootable

In order to boot from a hard drive, you'll need to mark a partition as active. Use the `flag` command and a partition number to do this.

```
fdisk: 1> flag 0
Partition 0 marked active.
```

Include this hard drive in your BIOS boot order, and the computer should try to boot from it. Simply marking a partition as active doesn't mean that the computer *can* boot from it; however, you will still need a kernel, boot loader, and all the other things that go into bootstrapping a computer.

To mark a partition as no longer active, delete and re-create it. (There is no `unflag` command.)

Exiting fdisk

Once you're satisfied with your work, enter **`quit`** or **`q`**, and `fdisk` should write the new MBR table to disk and exit. If you changed your mind, and don't want to make any changes, enter **`abort`** or **`exit`**, and `fdisk` should exit without saving changes to the MBR partition table.

Labeling Disks

OpenBSD uses `disklabel` to set up partitions on all hardware platforms. We used `disklabel(8)` as part of the installation process, but you need to partition new disks before you can use them. (You can also use `disklabel` to back up, restore, and duplicate partition tables.)

Viewing Labels

To view the current disklabel, just give the disk name as an argument. Here's how to see the disklabel of the empty disk from the previous section:

```
# disklabel wd1
❶ # /dev/rwd1c:
...
❷ duid: 0000000000000000
...
16 partitions:
#                size           offset  fstype [fsize bsize  cpg]
❸  c:         33554304                0  unused
```

This looks much like the disklabel we saw in Chapter 2, with a few critical differences.

First, note the device at ❶. The `disklabel` command accesses the raw device, but you should use the block device at the command line.

This label at ❷ has no DUID. This is the default empty disklabel. We will generate a DUID later.

At ❸, we see that this disk has only one partition, *c*, which represents the entire disk. You could create and use a filesystem on partition *c*, but it's not standard practice to do so.

Creating Disklabel Partitions

The simplest way to create partitions is to use the same interactive disklabel editor that we used to install OpenBSD. Give the disklabel editor the `-E` flag and the disk name:

```
# disklabel -E wd1
Label editor (enter '?' for help at any prompt)
>
```

Now you can add, remove, and edit partitions, just as in Chapter 3.

Throughout the rest of the book, we'll edit disklabels as needed to change partition and filesystem characteristics.

Backing Up and Restoring Disklabels

Before messing with a disk, back up its disklabel so that you can fall back to the old label if you screw up. You can back up the disklabel with this command:

```
# disklabel wd1 > wd1.disklabel.saved
```

To apply a saved disklabel to a disk, give `disklabel` the `-R` flag, the disk device, and the label file:

```
# disklabel -R wd1 wd1.disklabel.saved
```

This writes the saved label to the disk. You can use saved disklabels to duplicate partitioning across identical disks.

Now that you have partitions, let's put a filesystem on them.

The Fast File System

OpenBSD's filesystem, FFS, is an improved version of the filesystem shipped with BSD 4.4. FFS is sometimes called UFS (for Unix File System), and many system utilities still use UFS.[2]

FFS is designed to be fast, reliable, and able to handle the most common situations effectively while still supporting weird configurations. By default, OpenBSD tunes FFS for general use, but you can optimize it to fit your needs—whether you need to hold trillions of tiny files or a half dozen 30GB files. You don't need to know much about FFS internals, but you should at least understand blocks, fragments, and inodes.

FFS Versions

The original FFS was written in the 1980s and included hard-coded limits that were ample for the day. Filesystems could have up to 2^{31-1} blocks, or just under a terabyte (TB). In 1983, a 1TB filesystem was unthinkable. In 2013, 1TB drives are cheap.

For larger file systems, we have FFS version 2. FFS2 can support filesystems up to 8 zettabytes—unthinkable by 2013 standards. (FFS2 is likely to reach other limits before hitting the filesystem size limit, mind you.) OpenBSD supports both FFS and FFS2.

The i386 and amd64 boot floppies support only FFS, not FFS2. The installation CD, however, supports both. Most machines that need to boot from floppy don't need FFS2, and probably don't have a BIOS that can support 2TB drives anyway. The filesystem creation program `newfs(1)` is smart

2. OpenBSD is not the only operating system that still uses the BSD 4.4 filesystem or a descendant thereof. If a Unix vendor doesn't specifically tout its "improved and advanced" filesystem, it's almost certainly running a derivative of FFS.

enough to use FFS2 on filesystems large enough to need it, so for most installations, you don't need to worry about the difference between FFS and FFS2.

NOTE *In the exceedingly unlikely event that you actually require FFS2 on a machine that must be installed via floppy, be sure to format the critical system partitions of root (/), /var, and /usr as FFS, not FFS2. Use FFS2 only for partitions that are not critical to the system. Otherwise, you won't be able to use the installation disk for upgrades or emergency repairs.*

Blocks, Fragments, and Inodes

Both FFS and FFS2 are managed through blocks, fragments, and inodes. This arrangement isn't unique to FFS and FFS2; filesystems such as NTFS use data blocks and index nodes, too. The indexing system used by each filesystem is largely unique, however.

Blocks

Blocks are sections of disk that contain data. Files are placed in one or more blocks. OpenBSD's FFS uses a default block size of 16KB, or eight times the fragment size, whichever is smaller. Not all files are even multiples of 16KB, so leftover bits go in *fragments*. A fragment is one-eighth of the block size, or 2KB by default. A 20KB file fills one block and two fragments.

Inodes

Inodes, or index nodes, contain basic data about files, such as the file's size, permissions, and the list of blocks that contain the file. Collectively, the data in an inode is known as *metadata*, or data about data.

Superblocks

You'll also see references to *superblocks*, which are blocks that contain vital information about the filesystem's size and specifications. Superblocks are so important that FFS makes many backup copies of them. If you need to meddle with superblocks, you've probably done something wrong or your filesystem is FUBAR.

Creating FFS Filesystems

Use `newfs(8)` to create FFS and FFS2 filesystems and make sure that the disk has a disklabel. The `newfs` command takes one argument: the partition device node.

```
# newfs wd1a
/dev/rwd1a: 16383.9MB in 33554304 sectors of 512 bytes
81 cylinder groups of 202.47MB, 12958 blocks, 25984 inodes each
super-block backups (for fsck -b #) at:
```

```
 32, 414688, 829344, 1244000, 1658656, 2073312, 2487968, 2902624, 3317280,
3731936,
 ...
```

You'll see details about the filesystem size, how many blocks it includes, and so on. The location of each superblock backup is printed as `newfs` proceeds. (When computers and disks were much slower, this told the operator that the computer was actually doing something and hadn't seized up.)

The partition size determines which filesystem `newfs` uses. Partitions smaller than 1TB are formatted with FFS; larger partitions with FFS2. If you want to specify a particular filesystem format (yes, you can even specify the old-fashioned 4.3BSD format if you like), use the `-O` flag. It makes no sense to demand an FFS filesystem on a large partition, but you might have a reason to use FFS2 on a small partition.

```
# newfs -O 2 wd1a
```

If you think you need to specify which filesystem format to use on a new filesystem, you're probably wrong.

FFS Mount Options

OpenBSD can handle FFS partitions in several special ways, controlling what sorts of changes the filesystem supports and what sorts of files may exist. These are called *mount options*. You can specify mount options either when you mount partitions on the command line, as we'll discuss in "Mounting and Unmounting Partitions" on page 140, or in */etc/fstab*.

Mount Options and /etc/fstab

Specify a filesystem's mount options in a comma-separated list in the fourth field of the filesystem's */etc/fstab* entry. For example, here's an */etc/fstab* entry for the partition that contains my */home* directory:

```
244f6d3acd6374ad.k /home ffs rw,nodev,nosuid,softdep 1 2
```

I've specified the options `rw` (read-write), `nodev` (device nodes forbidden), `nosuid` (`setuid` programs forbidden), and `softdep` (soft updates). I'll cover these and other common mount options, and explain why you might want to use them.

Read-Only Mounts

If you only want to read the contents of a partition, and never write to it, you can mount the partition as *read-only*. In most cases, this is the safest way to mount a disk because you cannot alter the data on the disk or write any new data. If a filesystem should never change, mounting it read-only might make sense.

Read-only mounts are especially valuable when a particular filesystem is damaged. While OpenBSD won't let you perform a standard read-write

mount on a damaged or dirty filesystem, it can often mount those filesystems read-only. This gives you a chance to recover some data from the partition. (Not a large chance, but a chance.)

To mount a filesystem read-only, use the option `rdonly` or `ro`.

Read-Write Mounts

If you want to both read from and write to the disk, you'll want to mount the partition as *read-write*. By default, OpenBSD mounts all partitions as read-write.

Use the option `rw` to explicitly configure read-write mounts.

On modern hardware, I recommend using soft updates in conjunction with read-write mounts.

Synchronous Mounts

Using a synchronous mount is the safest way to mount a filesystem. OpenBSD can read data from a synchronous-mounted partition as fast as the hardware permits. Whenever you write to the disk, however, the kernel feeds a chunk of data to the disk, waits to receive confirmation that the disk has accepted the data and written it to disk, and then tells the program that requested the write that the data is now on disk.

You should know that even if you're using a synchronous mount, most hard drives lie about whether they have actually written the data to disk. These drives perform *write caching*, where writes are cached in a small flash or RAM buffer on the disk itself before the drive actually writes the data. This raises the question: Is a synchronous mount really synchronous? Hard drive vendors usually claim that in the event of a power failure, these disks retain just enough power to write the cache to disk.

Although they provide the greatest data integrity in the case of a crash, synchronous mounts are slow. You might use synchronous mounts when data integrity is crucial, but in most cases, it's overkill and you have little ability to verify that the mount is truly synchronous.

Activate synchronous mounts with the `sync` keyword.

Asynchronous Mounts

To write data quickly, but with a higher risk of data loss, mount partitions asynchronously. When using asynchronous mounts, the kernel informs software that all disk writes are successful before the disk confirms that the data was written. This is fast, but a system failure can leave inconsistent data on your disk.

Asynchronous mounts are useful when restoring a filesystem from backup, because if you get a power failure halfway through the restore procedure, you'll need to start over anyway. Don't use asynchronous mounts in production if you care about your data or would object to re-creating the filesystem.

Activate asynchronous mounts with the `async` keyword.

Soft Update Mounts

Soft update mounts organize and arrange disk writes so that filesystem metadata remains consistent at all times. This gives performance similar to that of an asynchronous mount with the reliability of a synchronous mount. While that doesn't mean that all data will be written to disk—a power failure at the wrong moment will result in lost data—using soft updates prevents a lot of filesystem integrity problems caused by that lost data. It's not the default because some older, smaller hardware doesn't have enough memory to support it, but if you're using modern i386 and amd64 hardware, I recommend enabling soft updates for all FFS partitions.

To mount a filesystem with soft updates, use the `softdep` option.

"Don't Track Access Time" Mounts

FFS records the last time a file was read, executed, or otherwise viewed. Updating these access times consumes a small but measurable amount of disk I/O and performance. You can use the `noatime` mount option to tell OpenBSD to not update the access time on any file.

Using `noatime` makes sense on laptops, where minimizing power usage is critical. If you're tempted to use this option on your server to squeeze out a little extra performance, you should buy a faster disk instead. Some software, such as the Mutt mail client, will break if run on filesystems mounted `noatime`.

No Device Nodes Permitted Mount

By using the `nodev` mount option, you can tell OpenBSD to not interpret any device nodes on any given filesystem. Intruders can try to create "rogue" device nodes and use them to write files or attack the network, but if the kernel won't recognize those device nodes, it cuts off this whole category of attacks.

This type of mount is also useful if you have hard drives from multiple operating systems on your computer. For example, if you dual-boot OpenBSD and Linux on your computer, but you don't want to accidentally access a Linux device node when using OpenBSD, the `nodev` option will prevent you from doing so. (You might think you would notice that you had typed */linux/dev/hda* rather than */dev/wd1*, but never underestimate your ability to screw up.) In most cases, the partition containing */dev* is the only one that should contain device nodes.

Execution Forbidden Mounts

The `noexec` mount option prevents any binaries on the partition from being executed. Mounting */home* with the `noexec` option helps prevent users from installing and running their own programs, but for it to be effective, you'll need to make sure users can't install binaries in any shared areas, such as */tmp* and */var/tmp*.

Note that forbidding execution of binaries doesn't prevent users from running interpreted scripts from that partition. Maybe the users can't run a compiled C program, but if they can run `perl $HOME/rootkit.pl`, then `noexec` won't slow them down very much.

setuid Forbidden

The `nosuid` option disallows `setuid` behavior from programs on this filesystem. Many partitions should not have `setuid` files, and setting this is an easy way to disrupt them. OpenBSD sets this on partitions such as */home* and */tmp* by default. You must carefully place this option on all user-writable filesystems for it to prevent undesired behavior.

Do Not Automatically Mount This Filesystem

`noauto` isn't actually a mount option, but rather a way of telling OpenBSD to not mount a given partition listed in */etc/fstab* at boot. I frequently make */etc/fstab* entries for removable media drives, but the system should not try to mount these at boot. The boot will hang if a partition required by */etc/fstab* is not available, and I don't want my computer to refuse to boot just because I unplugged a flash drive.

Filesystem Integrity

Both versions of FFS go to a great deal of trouble to ensure that the data on disk is correct and intact. The blocks that contain a file should be recorded in an inode, the inodes should all be referenced by directory entries, and so on. When you remove a file, all references to that file should be removed.

After a system failure, however, data might not be consistent. Metadata might reference blocks that were previously erased; a file might be in a different location than the inode record specifies; and the filesystem might have all kinds of references pointing to things that have moved, changed, or disappeared. These inconsistent, or *dirty*, filesystems cannot be trusted and must be rationalized, or *cleaned*, before you can mount them read-write. If you mount a dirty filesystem read-only, it might only panic your system, but if you force OpenBSD to mount a dirty filesystem read-write, you will damage the dirty filesystem even more.

At boot, OpenBSD performs a minimal inspection and cleaning, or *preening*, of the filesystems and will automatically correct any minor problems found. If preening cannot fully clean the filesystem, the boot will hang until you intervene.

When confronted with a dirty filesystem, you have a few options: use the filesystem checking tool `fsck(8)`, debug the filesystem with `fsdb(8)` and `clri(8)`, or throw the filesystem away and run `newfs(8)`. Most of the time, you'll attempt to repair the filesystem with `fsck`. Using `fsdb` successfully requires more knowledge about FFS innards than I possess, so I recommend it to only those who really want to develop an in-depth knowledge of FFS and have

a whole bunch of time to devote to it. Rebuilding the filesystem with `newfs` destroys everything on the filesystem, but it's a decent choice for partitions that contain only ephemeral data, such as */usr/obj*.

You can use `dump(8)` to copy the damaged filesystem before trying any of the repairs. This gives you the option to fall back to the current state if attempts at repairing the disk fail. (If you have to do this, though, you should probably reevaluate your backup strategy.)

Running fsck

If you try to mount a dirty filesystem either at boot time or during routine operation, you'll see a message that looks like this:

```
/dev/rwd1a: UNEXPECTED INCONSISTENCY; RUN fsck_ffs MANUALLY
```

The `fsck(8)` program is a frontend for several filesystem-specific integrity-checking programs. When you run it, `fsck` identifies the type of filesystem and calls the correct integrity checker for you. Run `fsck` by giving it the device name of the filesystem you want to check:

```
# fsck /dev/wd1a
```

You can use either the raw or cooked device name; `fsck` is smart enough to use the raw node even if you give the cooked device name.

Examining the filesystem can take quite a while, so be patient.

When run on a dirty filesystem, `fsck` will probably find a number of problems: blocks that have become disassociated from their inodes, inodes that reference empty blocks, and so on. It can often make a good guess as to how everything fits together.

When `fsck` finds a problem that it isn't absolutely sure about, it will suggest a fix and ask if you want to make the change. If you answer `y`, `fsck` makes the change. If you answer `n`, `fsck` leaves the filesystem unchanged. If you tell `fsck` not to make the change it suggests, the filesystem will still be dirty, and you'll need to fire up `fsdb` or `clri` and make the change you think more appropriate.

Sometimes, `fsck` can't identify the name or directory of a file recovered from a damaged filesystem. These files go into the partition's *lost+found* directory (for example, */usr/lost+found*). You'll need to use programs such as `grep` and `strings` to try to identify these files by their contents.

Blindly Trusting fsck

Those of us who lack the skills to debug a filesystem find ourselves in a difficult situation, where we can either accept that `fsck(8)` knows what's best or just restore from backup. If your filesystem was performing a lot of disk I/O just before system failure, `fsck` might need to make dozens or hundreds of changes. You could spend an hour sitting at the console pressing `y` repeatedly.

If you decide to trust `fsck` and hope it's right, run `fsck -y`. This means "answer `y` to every question." You might wind up with the entire contents of the filesystem in the *lost+found* directory, or you might lose every file on the filesystem. But unless you're intimately familiar with the innards of FFS, you would need to restore from backup anyway.

If you run `fsck` and realize partway through that you would like to answer `y` to all the questions that follow, enter `F`. That tells `fsck` to answer `y` to all remaining questions.

At the end of the procedure, you've either recovered your system or need to restore from backup.

What's Currently Mounted?

While performing routine work, inevitably you'll need to check which disks are currently mounted and which are not. To see a list of all mounted filesystems and their mount options, run `mount(8)` without any options:

```
$ mount
/dev/wd0a on / type ffs (local)
/dev/wd0k on /home type ffs (local, nodev, nosuid)
/dev/wd0d on /tmp type ffs (local, nodev, nosuid)
/dev/wd0f on /usr type ffs (local, nodev)
/dev/wd0g on /usr/X11R6 type ffs (local, nodev)
/dev/wd0h on /usr/local type ffs (local, nodev)
/dev/wd0j on /usr/obj type ffs (local, nodev, nosuid)
/dev/wd0i on /usr/src type ffs (local, nodev, nosuid)
/dev/wd0e on /var type ffs (local, nodev, nosuid)
```

Both FFS and FFS2 partitions show up as `type ffs`. The word `local` means that the partition is on a physical drive attached to this machine. We covered the various mount options (`nodev`, `nosuid`, and so on) earlier in this chapter.

Note that `mount` displays the device node mounted at each partition, not the DUID. If you want to see the DUID of a disk, check the disklabel.

Mounting and Unmounting Partitions

To attach filesystems to your directory tree, or *mount* them, use `mount(8)`. If you've never manually mounted filesystems before, boot your OpenBSD machine into single-user mode (see Chapter 5) and follow along.

In single-user mode, OpenBSD mounts only one partition: the root partition, which it mounts read-only. The root partition contains just enough of the system to perform basic setup, establish core services, and find the other filesystems.

Because filesystems other than the root are not mounted, their content is not accessible. Look in, say, */usr* on a system in single-user mode, and you'll find that it's empty. OpenBSD hasn't lost the files; it just hasn't mounted the partition containing those files.

To get any real work done in single-user mode, you probably need to mount other filesystems.

Mounting Standard Filesystems

To manually mount a single filesystem listed in */etc/fstab*, give `mount(8)` the name of the filesystem you want to mount. Here, we'll mount our */usr* partition:

```
# mount /usr
```

This mounts the partition exactly as described in */etc/fstab*, with all the options specified therein.

To mount all of the partitions listed in */etc/fstab*, give `mount` the `-a` flag:

```
# mount -a
```

All of your filesystems (except those not listed in */etc/fstab* and those with the `noauto` option) should now be mounted.

Mounting at Nonstandard Locations

Perhaps you must mount a filesystem at a location not specified in */etc/fstab*. I do this most commonly when adding a disk to a machine. To mount a partition at a location other than that specified in */etc/fstab*, or to mount a partition without an */etc/fstab* entry, give the partition device name and the mount point.

```
# mount /dev/sd0d /mnt
```

You must use the full path for the device node, not just the brief device node name.

Instead of the path to the device node, you could use the DUID, a period, and the partition letter, but on the command line, that's more painful than using the path to the device node.

Unmounting Partitions

To disconnect a filesystem from the directory tree, use `umount(8)` on a mount point. (Note that there is only one `n` in this command.) Here, we'll use `umount` to unmount our */usr* partition:

```
# umount /usr
```

You cannot unmount filesystems that are in use by any program. Even a command prompt in the mounted directory will prevent you from unmounting the partition.

To unmount all partitions except the root partition, pass `umount` the `-a` flag:

```
# umount -a
```

As programs almost certainly have files open on every partition, this probably works only in single-user mode. Note that you don't need to unmount all partitions to leave single-user mode.

Mounting with Options

Suppose you pull a disk from a decommissioned OpenBSD machine and you need to retrieve some files from it. You want to mount the disk read-only so that you don't change any of the files on the disk. To manually mount a partition with options not specified in */etc/fstab*, use the `-o` flag.

For example, if the disk shows up as */dev/sd0* and you want to mount partition *a*, run this command:

```
# mount -o ro /dev/sd0a /mnt
```

To prevent old software from running on your newer system, it might be a good idea to use some of the options we covered earlier, such as `noexec`, `nodev`, and `nosuid`.

How Full Is That Partition?

To get an idea how much free space remains on your partitions, use `df(1)`. This program displays the total number of filesystem blocks on each partition, how many blocks are in use, and how many blocks are free. It also gives you the percent in use.

One annoying thing about `df` is that it offers this information in 512-byte blocks by default. This was fine when disks were much smaller, but today, it's like measuring the distance of an airplane flight in yards. Some people have done this for so long that they automatically perform block transformations in the back of their mind.[3] For the rest of us, the `-h` flag tells `df` to provide human-readable output, such as megabytes or gigabytes, giving us something like this:

```
# df -h
Filesystem     Size    Used   Avail Capacity  Mounted on
/dev/sd0a     1005M   39.1M    916M     4%    /
/dev/sd0k     26.9G   27.0G   -104M    -1%    /home
/dev/sd0d      3.5G   12.0K    3.3G     0%    /tmp
...
```

You might wonder why the */home* partition in this example has negative free space. How is that possible? By default, FFS reserves 5 percent of each partition for moving files and reducing fragmentation. When you exceed 100 percent disk utilization, you begin tapping into this reserved space.

FFS performance degrades quickly when the partition is overfull. It's best to keep some free space on your disk so that FFS can defragment itself.

3. Hi, Henning!

You can reduce the amount of space FFS reserves, but doing so will impact performance. See `tunefs(8)` for instructions on how to shoot yourself in the foot.

What's All That Stuff?

When you see a partition is full, the obvious question is "What's filling up my disk?" Every hard drive I've ever owned has gradually filled up for no apparent reason. You can identify individual large files with `ls -l`, but recursively examining every directory in the filesystem is impractical and tedious (not to mention annoying).

To check the number of filesystem blocks used within each directory below the current directory, use `du(1)`.

```
$ du
164     ./.ssh
2       ./old
6       ./.mozilla/firefox/bcpuv16e.default/chrome
80      ./.mozilla/firefox/bcpuv16e.default/Cache/0/B0
354     ./.mozilla/firefox/bcpuv16e.default/Cache/0/31
28      ./.mozilla/firefox/bcpuv16e.default/Cache/0/7A
...
```

When I run `du` in my home directory, I get 700 entries; of those, 563 are related to some Mozilla tool. This kind of list intimidates the new sysadmin and makes the experienced sysadmin work too hard. Rather than cull through this list manually, tell `du` to show only the total for directories in the current directory, and then sort the output so that the largest directories appear first.

```
$ du -s * | sort -rnk 1
25224805  Dark_Shadows_Complete_Series
141104    mibs
14948     tarballs
4668      work
1864      pix
...
```

I now know why my */home* partition is full.

You can tell `du` to display human-readable values with the `-h` flag, but doing so will show values in a mix of gigabytes, megabytes, and kilobytes, making `sort` far less useful.

Setting $BLOCKSIZE

Many disk tools—including, but not limited to, `du(1)` and `df(1)`—display information in 512-byte blocks. If you're accustomed to working in blocks, you probably won't mind seeing them. If you're not used to working in blocks, however, they'll probably make you want to tear out your hair.

The environment variable BLOCKSIZE tells these programs to display information using blocks of a different size. If you set BLOCKSIZE to K, df and du will display totals in kilobytes. If you set it to M, these tools will show megabytes instead. Check your shell manual page or the dotfiles in your home directory for examples of setting environment variables.

Adding New Hard Disks

The OpenBSD installer walks you through formatting and partitioning your initial hard disks. If you need to add a disk to an existing system, however, you must run these commands yourself. The good news is that if you can install OpenBSD, you already know how to use the commands, and the only hard part is learning which commands to run.

I'll show you how to move */home* to a new disk as an example. You could create a new partition on your existing disk if you have some empty space, but that would eliminate the need for this example, so I'm going to pretend I never gave you that advice. (Also, moving partitions to a separate disk controller channel will improve performance.)

WARNING *Before touching anything involving disk partitioning or filesystems, back up your system. Verify that backup before starting. You have been warned.*

Creating an MBR Partition

The i386 and amd64 platforms require disks to have MBR partitions as well as OpenBSD partitions. A standard new disk needs a single OpenBSD MBR partition covering the entire disk. Passing the -i argument to fdisk does exactly this. Let's create a new MBR partition on wd1, our new disk:

```
# fdisk -i wd1
Do you wish to write new MBR and partition table? [n] y
Writing MBR at offset 0.
```

Once you have an MBR partition on your disk, you can create disklabel partitions.

Creating a Disklabel

All OpenBSD platforms use disklabel partitions. To activate the same disklabel editor we used during the install process, give disklabel the -E flag and the disk name:

```
# disklabel -E wd1
```

This should look familiar from earlier in this chapter. Use the interactive disklabel editor to create your new partitions. For a single */home* directory, we'll use one large partition, *wd1a*. The new label should look like this:

```
#               size           offset  fstype [fsize bsize  cpg]
  a:        33543648               64  4.2BSD   2048 16384    1
  c:        33554304                0  unused
```

When you've finished editing partitions, check your work by printing the disklabel. This should also give you the DUID of the new disk.

When you're satisfied with the partitioning, use `newfs` to create a filesystem on the new partition:

```
# newfs wd1a
```

You're now ready to add the filesystem to your computer.

Moving Partitions

Moving data from one disk to another is slightly more complex than adding new partitions. You must first mount the new drive in a temporary location, copy files to that location, remove them from the old location, and mount the new drive in its previous home.

Our new */home* filesystem is on disk partition *wd1a*. The default "temporary mount" location is */mnt*, so mount it there. This is strictly temporary, so there's no need to mount it via the DUID or make an */etc/fstab* entry for this.

```
# mount wd1a /mnt
```

You can then use `tar(1)`, `cpio(1)`, or `dump(8)` and `restore(8)` to copy the files to the temporary location. Here, we copy everything in */home* to */mnt*.

```
# (cd /home && tar cf - . ) | (cd /mnt && tar xpf - )
```

You could also use `cp(1)` or `mv(1)` for this, but these commands don't guarantee that file permissions and ownership will copy intact. OpenBSD's versions of these programs have never given me errors when I copy or move files, but I've learned from other Unix-like operating systems that `tar` and `cpio` are both more reliable when moving entire file hierarchies. If you're using file flags for security (see Chapter 10), you must use `dump(8)` and `restore(8)` to retain those flags.

Using `tar` or `cpio` does not delete files from their original location. This means that if a user changes files in his home directory after you copy them but before you change the mount point, he will lose his changes as you shuffle disks around. [4]

Now update */etc/fstab* to reflect your new disk.

4. Presumably you warn your users before doing maintenance. Or at least *during* maintenance. Or . . . maybe afterward.

Adding New Filesystems

Look at the disklabel for the new disk and get the disk's DUID. This new disk has a DUID of `fea9194ee78362d8`. Use the DUID and the partition letter to make an */etc/fstab* entry for your new partitions.

```
fea9194ee78362d8.a /home ffs rw,nodev,nosuid,softdep 1 2
```

You might want to keep the old partition available at a new location, such as */oldhome*.

If you're not sure about the mount options to use for your new partitions, the options `nodev`, `nosuid`, and `softdep` are generally safe. You probably want the partition mounted read-write (`rw`) as well.

Now unmount the old and mount the new.

```
# umount /home
# mount /oldhome
# mount /home
```

When you unmount a partition, `umount` doesn't check */etc/fstab*. You tell it to unmount a partition, and it unmounts that partition.

Stackable Mounts

OpenBSD filesystems are *stackable*, which means that you can mount one partition over another. The partition on top hides any files in the filesystem below.

Look at your system in single-user mode. By default, only the root partition is mounted. You can go look in the */home* directory, and it will be empty. There's no reason you can't put files in the */home* directory, even when */home* isn't mounted. Suppose you copy a couple of core files into */home* while in single-user mode, and then go into multiuser mode. All the usual partitions are mounted. If you then look in */home*, you won't find your core files.

What happened? Where did those files go?

The files are in the directory */home*, but on the root partition. The */home* partition is mounted above that directory, so the */home* partition obscures the files in the */home* directory on the root partition. To access those hidden files, you must unmount the */home* partition. Those hidden files continue to take up space on the root partition, however.

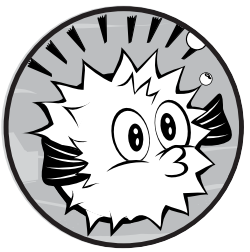
This happens more commonly when splitting a partition. For example, if you find that your */var* partition is too small, you might move */var/www* into its own partition on a separate disk. To free up space on the original */var*, delete the files you copied to */var/www*.

With the basics of filesystem management under your belt, you're now ready to look at some of OpenBSD's more interesting filesystem tricks.

9

MORE FILESYSTEMS

Encrypt your hard drive?
Software RAID can save your day,
or ruin your life.



Disk management isn't complicated, but there's enough material that it gets two chapters. Lucky you! In this chapter, we'll start with how to automatically back up your vital root partition to a second disk. Then we'll explore how OpenBSD can use additional memory as disk space via a memory filesystem and how to set that up. Next, we'll access disks formatted for other operating systems, such as NTFS, ext2, and FAT. Removable media isn't difficult to work with, but has its own concerns. If you don't need the actual media, but can work with disk images, you can access those. Both topics are covered in this chapter. We'll also discuss using NFS, as both a server and a client. Our final topic is OpenBSD's disk redundancy and disk encryption features.

Backing Up to the /altroot Partition

You can lose most of your partitions and still hope to recover the rest of the system. If you lose your root filesystem, however, recovery becomes a much more difficult task. While you could back up critical files from */etc* and use them to restore your system, OpenBSD provides the */altroot* partition as an easy way to automatically duplicate the root partition on a second disk.

An emergency root partition on a second disk gives you an easier path to recovery in the event of a disk failure. Booting to the second disk lets you pull any data off that disk, and possibly even from undamaged partitions on the first disk, before replacing the failed disk. There's no reason to back up your root partition to the same disk, however, as the whole disk will probably be unusable.

This backup requires a free disk partition the same size as your root partition, located on a different disk. The OpenBSD installer defaults assume that you have only one disk. If you have a second disk during installation, you need to use a custom install process to create the */altroot* partition, as we did in the multiple disk installation in Chapter 3. While configuring partitions is easiest during the installation, you can add more disks later if needed, as discussed in Chapter 8.

Your */altroot* partition needs an */etc/fstab* entry. If you created the partition during the install process, that */etc/fstab* entry already exists but has the wrong mount type. If you created this partition after installation, you'll need to create an */etc/fstab* entry yourself. The */altroot* partition needs a mount type of `xx`, as shown here:

```
a914f9a264fa64e6.a /altroot ffs xx 0 0
```

You cannot mount this partition from its */etc/fstab* entry, as `xx` is not a valid mount type. (You could run, say, `mount /dev/sd1a /altroot` if you want to manually mount this partition.) The daily system maintenance job */etc/daily* uses this mount option to identify the root backup partition.

To enable the */altroot* backup, add `ROOTBACKUP=1` to your */etc/daily.local* file.

Memory Filesystems

In addition to creating partitions on raw disk, OpenBSD lets you create partitions in system memory. A *memory filesystem* (*MFS*), or *memory disk*, lives in your machine's RAM, rather than on a physical disk. Reading and writing files to and from such a filesystem is much faster than accessing those same files on a spinning disk, which makes a memory-backed filesystem a huge optimization for certain applications.

If MFSs sound too good to be true for high-performance environments, that's because they are. Understand their limits before you implement them everywhere. First, RAM does not persist across reboots or shutdowns, so either will erase the contents of an MFS. While this might seem obvious,

I've surprised myself more than once by losing a file stored on a filesystem I had forgotten was an MFS. Furthermore, if your system crashes, you'll lose any data stored on an MFS.

You can use an MFS partition as scratch space to rapidly compile, compress, decompress, or otherwise manipulate temporary files. I've seen news server histories, database locks, and other application-specific files stored on MFSs.

An MFS works even in situations where the system regularly swaps. The kernel retains any information being actively used in memory, while transferring unused information to swap space. This is excellent for small partitions like */tmp*, in which small, frequently used files can be quickly accessed. Files that are less frequently accessed end up in swap space, which gives performance similar to accessing a physical disk.

One last word of caution: Don't make heavy use of MFSs if you don't have RAM to spare. If you run short on combined memory and swap space, your system will perform very poorly.

Creating MFS Partitions

Create temporary MFS partitions with `mount_mfs(8)`. Like other `mount_` commands, `mount_mfs` takes two arguments: the physical device and a mount point. Unlike physical disks, memory doesn't have a device node, so use the device node of the system swap space. If you have multiple swap partitions, pick whichever you like.

Here is how you can create a memory-backed filesystem by passing a swap partition, */dev/sd0b*, and a desired mount point, */mnt*, as arguments to `mount_mfs`:

```
# mount_mfs /dev/sd0b /mnt
```

The size of this partition will be limited only by the size of your swap partition.

You can create smaller memory-backed filesystems, so that you will have memory and/or swap space available if you fill the memory disk. Specify the size with the `-s` flag and a number of sectors, or with a trailing `b` (bytes), `m` (megabytes), or `g` (gigabytes). Here's how to create a 128MB MFS on */mnt*:

```
# mount_mfs -s 128m /dev/sd0b /mnt
```

If you request an MFS larger than your system can support, you'll get a warning like `mmap: Cannot allocate memory`. Try again, this time with a more reasonable size.

Mounting an MFS at Boot

You can mount an MFS at boot by adding an */etc/fstab* entry. You only need a mount point and the partition size.

```
❶swap    ❷/mnt    ❸mfs    ❹rw,async,-s=128m    ❺0    ❻0
```

You don't need to specify a specific swap device; OpenBSD is smart enough to let you say the memory disk is generically swap-backed ❶. Just as with any other partition, you also need to specify the mount point ❷ and the filesystem type ❸.

When dealing with a memory disk, you can use different options than you would for a traditional disk ❹. Since a system crash would destroy all files on the MFS anyway, you can safely mount an MFS partition as asynchronous using the `async` option. You might also want to use `nodev` and `nosuid` mount options on this partition. You can specify the size with the `-s` option, but make sure that you put an equal sign (`=`) between the `-s` and the size. Because */etc/fstab* uses whitespace to separate fields, OpenBSD will think the dump level is 128m if you don't use an equal sign.[1]

Data on a memory disk is by definition disposable, so don't back it up ❺. Similarly, never use `fsck(8)` with a memory disk at boot ❻. The memory disk is created anew at each boot, so it is automatically internally consistent.

Foreign Filesystems

Any partition that uses a non-FFS filesystem is foreign to OpenBSD. Although OpenBSD can access many foreign filesystems, don't expect it to be seamless.

Support for some filesystems is incomplete. For example, you can mount Microsoft NTFS partitions only as read-only. Other filesystems don't support the full range of OpenBSD commands. Because FAT filesystems don't have any concept of file ownership or permissions, commands like `chmod` and `chown` won't change anything on the disk.

Each supported filesystem has its own mount program to handle the vagaries of that filesystem. To simplify your life, `mount` can usually recognize supported filesystems from the on-disk format and call the correct mount program as needed. To mount a foreign filesystem, you need the device node and a mount point. Depending on the filesystem, you may also need to know the type of filesystem you'll be mounting.

Inodes vs. Vnodes

Before we talk about foreign filesystems, let's touch on something that confused me for a long time: the difference between inodes and vnodes.

The FFS uses index nodes, or *inodes*, to map blocks of disk that contain data. This worked just dandy when hard drives were big, expensive things that no one moved between computers. Over the years, however, swapping disks between machines has become more popular.

Although Unix-like systems think in terms of accessing files via inodes, the FAT32 filesystem doesn't use inodes, ext2fs's inodes don't map directly onto FFS inodes, and CDs use a completely different layout. To access all of these filesystems in a consistent way, BSD needed another layer of abstraction.

1. I don't know what a dump level of 128m means, other than "not what I want."

The virtual node, or *vnode*, is an abstraction layer the kernel uses to access all filesystems. Users never manipulate vnodes directly, but you'll see references to them throughout OpenBSD's documentation. Every tool that reads or writes to disks does so through vnodes, which map the requests to the filesystem. When you write to an FFS block or inode, the kernel addresses data to a vnode, which in turn maps to an inode. When you write to a FAT32 filesystem, the kernel addresses data to a vnode mapped to a point in the FAT32 filesystem. You use inodes only when dealing with FFS systems, but your data will pass through a vnode when accessing any filesystem.

Don't let references to vnodes on non-FFS systems confuse you. They're part of OpenBSD, not the filesystem.

Common Foreign Filesystems

Common foreign filesystems include MS-DOS, NTFS, ext2fs, and CD. We'll look at how to access disks formatted for those operating systems with OpenBSD.

MS-DOS

OpenBSD supports the FAT, FAT16, and FAT32 filesystems. These formats are commonly found on flash media, old Microsoft operating systems, and floppy disks.

To mount a filesystem with a FAT filesystem partition, use `mount_msdos(8)`.

```
# mount_msdos /dev/sd3i /mnt
```

Not sure which partition on the disk is the FAT filesystem? Run `disklabel(8)` on the drive and see. FAT filesystems are often located on the *i* partition. And even if you try inserting your USB drive and mounting its *i* partition, OpenBSD will probably figure out that it's a FAT system.

If you work with FAT disks often, you might investigate */usr/ports/sysutils/mtools*, a collection of software for working with FAT filesystems without mounting them. While `mount_msdos` is quite reliable, mtools offers a more elegant interface.

NTFS

To mount disks formatted for modern Microsoft operating systems, use `mount_ntfs(8)`.

```
# mount_ntfs /dev/sd3k /mnt
```

As I write this, OpenBSD supports NTFS4 (from Windows NT) and NTFS5 (in Windows 2000 and XP). Windows Vista and newer systems are not yet supported, but they might be by the time you read this.

If you need to view file attributes specific to the NTFS filesystem, check the `mount_ntfs` man page for details.

ext2fs

To mount ext2fs and ext3fs filesystems, use `mount_ext2fs(8)`. (The one program mounts both types of filesystem.)

```
# mount_ext2fs /dev/sd3l /mnt
```

Owing to their shared Unix heritage, the Linux ext2fs and ext3fs filesystems support many FFS-like features. Unlike with NTFS, you can safely read and write ext2fs and ext3fs disks in OpenBSD. You cannot, however, read ext4fs partitions using OpenBSD.

CD

Compact discs formatted for data use the ISO-9660 filesystem. To mount a CD, use `mount_cd9660(8)`.

```
# mount_cd9660 /dev/cd0a /mnt
```

Mount CDs using either the *a* or *c* partition on the device. If you would like to save yourself a few keystrokes, `mount(8)` is very good at automatically detecting ISO-9660 filesystems. The device node for a CD is tied to the CD drive, not the disk itself, so the node shouldn't change unless you add another drive.

If you're interested in burning a CD, look at `mkhybrid(8)` and `cdio(1)`.

Foreign Filesystem Ownership

Most foreign filesystems either have no concept of file ownership or have an ownership scheme incompatible with that of Unix-like operating systems. (Notable among these filesystems are FAT and NTFS.) The programs that mount these kinds of filesystems thoughtfully allow you to specify the ownership of files on the filesystem. The `-u` flag lets you specify a file owner, and the `-g` flag lets you specify the group.

For example, here's how I would mount a FAT filesystem as owned by my account:

```
# mount_msdos -u mwlucas -g mwlucas /dev/sd3c /mnt
```

Some other filesystems use permissions schemes compatible with OpenBSD's permissions. For example, all of the information OpenBSD needs to assign permissions to files and directories is contained within an ext2fs filesystem. That doesn't mean that an ext2fs filesystem will perform seamlessly on OpenBSD, however. Though OpenBSD will respect the ext2fs disk's permissions, the user ID numbers probably won't match up between the operating systems.

Removable Media

These days, the removable media you'll most likely deal with are external hard drives, flash drives, and CDs. The CD is the simplest, because you know how to use `mount(8)` and `umount(8)`, and you know its device node and filesystem type will always be the same. But how do you identify the device name of a removable hard drive?

When you attach a drive to your machine, OpenBSD automatically assigns your drive a device node to your console and prints a message to the console. You can check the console as you attach the drive, or you can watch your messages log by running `tail -f /var/log/messages` before attaching the drive.

If you frequently use a particular removable disk, you can simplify your routine by making an */etc/fstab* entry for it. Here are some sample */etc/fstab* entries for a CD and a FAT flash drive.

```
/dev/cd0c /cdrom cd9660 ro,noauto
/dev/sd3i /mnt msdos rw,noauto
```

You can't use DUIDs for removable media, because the actual media might change.

Now you can mount your CD on */cdrom* by entering `mount /cdrom`, and your FAT flash drive on */mnt* by entering `mount /mnt`.

Note that OpenBSD does not create a */cdrom* directory by default; you'll need to create it yourself. You could point both of these at */mnt*, but I like having a dedicated CD mount point on my systems, and having two devices share a mount point risks concealing one of the filesystems. (Remember that OpenBSD has stackable mounts, as discussed in Chapter 8.)

Mounting Filesystem Images

You can mount a disk image and access the image just as you would a disk partition. This is very useful for those times you want to extract a few files from an ISO but don't want to bother burning the image to physical media. The trick to mounting a disk image is attaching the image to a device node so that you can use the proper `mount` command.

OpenBSD uses the `vnconfig(8)` program to attach disk images to device nodes. (Remember that a vnode is an abstraction layer between the kernel and a filesystem.) Use `vnconfig` to "wire" vnodes between a file and a device node, and then access them through OpenBSD's */dev/svnd* devices. Depending on the disk image type, the image might have MBR partitions, disklabel partitions, or just a filesystem.

The default kernel has four vnode devices. If you need to mount more than four disk images simultaneously, edit your kernel binary using `config(8)`'s `-e` option, as discussed in Chapter 18.

Attaching Vnode Devices to Disk Images

The `vnconfig(8)` command takes two arguments: the device node you want to use and the disk image you want to mount.

```
# vnconfig /dev/svndXc /path/to/file
```

Note that this example uses the *c* partition of the device. This allows you to treat the disk image as a whole disk.

Suppose you have an ISO image named *install52.iso* that you would like to mount. First, use `vnconfig` to attach this image to vnode device 0.

```
# vnconfig /dev/vnd0c install52.iso
```

You can then use `mount` to attach the vnode to an */mnt* directory.

```
# mount /dev/vnd0c /mnt/
```

OpenBSD's `mount(8)` is smart enough to recognize this as a CD filesystem and mount it as such. If you're mounting a disk image that uses a less detectable filesystem, you need to use the specific `mount` command for that filesystem.

Detaching Vnode Devices from Images

Vnode devices attached to a file remain attached until specifically disconnected, and you can attach a vnode device to only one file at a time. To disconnect the vnode device from the file, use the `-u` flag with `vnconfig`. For example, to disconnect the vnode device located at *vnd0c*, run this command:

```
# vnconfig -u vnd0c
```

You can now attach this vnode device to another file.

Using the full path to the device is optional in `vnconfig`. If you know the device name, you can use it without the leading */dev*, as in the preceding example.

Basic NFS Setup

NFS allows one machine to access files on another machine. NFS has its origins in UNIX, but today appears in most operating systems, including those from Microsoft and Apple. OpenBSD supports NFS versions 1 through 3 as both a client and a server.

Entire books can be—and have been—written about NFS. We won't go into the intimate details of NFS, but rather focus on getting a basic NFS share working on OpenBSD. Configuring NFS the first time can be intimidating, but after setting up a file share or two, you'll find it straightforward.

If you have a complicated NFS environment—involving multiple versions of multiple operating systems—or if you want to share a directory among hundreds of active clients, you should do further research, but even a basic setup will help to simplify parts of your job.

NFS works on the client/server model. One computer, the server, offers filesystems to other computers. The server is *exporting* a filesystem, and the filesystems on offer are called *exports*. NFS clients can mount exports in a manner almost identical to that used to mount local filesystems.

One important thing to remember about NFS is that it is *stateless*, which means that NFS does not track the condition of a connection. You can reboot an NFS server, and the client won't throw a fit. The client cannot access files on the server while the server is down, but once the server returns, the client will pick up right where things left off. Other network filesystems are not always so resilient. Statelessness causes its own problems as well. For example, clients cannot know when a file they are currently reading has been modified by another client.

If you're just learning NFS (or OpenBSD's implementation of NFS), check */var/log/messages* for NFS-related error messages. If you've repeatedly reconfigured your NFS server as part of learning, and things just don't work correctly, reboot your NFS server and/or client. NFS is complicated, and sometimes starting with a clean stack clears up a lot of problems. Once you understand how all the pieces fit together, a reboot to resolve problems should never be necessary.

NOTE *The NFS protocol has evolved over the years, and every operating system has implemented a slightly different version of NFS. Other BSDs, Illumos, Linux, Apple, Microsoft, and most other operating systems can work with OpenBSD's NFS support, but each may require an occasional tweak for specific environments. If you're having trouble getting NFS to work with OpenBSD and another operating system, read* `mount_nfs(8)` *and feed the details to your favorite search engine. The odds that someone else has experienced this problem before are good.*

The OpenBSD NFS Server

By default, OpenBSD includes all the programs necessary to act as an NFS server, but you must turn it on. The NFS server requires three daemons:

`portmap(8)` Maps requests for remote procedure call (RPC) services to TCP/IP port numbers.

`mountd(8)` Listens for incoming NFS mount requests.

`nfsd(8)` Processes requests for filesystem actions.

The `portmap(8)` daemon has its own *rc.conf* flag, as it can be used by many other RPC services. The `mountd(8)` and `nfsd(8)` daemons are controlled by a single *rc.conf* flag.

Add the following entries to *rc.conf.local* to start all three processes at boot time:

```
portmap=YES
nfs_server=YES
```

You can start these three daemons from scripts in */etc/rc.d*. If you try to start these daemons now, however, they won't run. You must configure at least one export before the NFS server daemons will start.

Exporting Filesystems

To export filesystems, define which clients may mount which filesystems and/or directories in */etc/exports*. This file takes a separate line for each disk device on the server and each client or group of clients that can access that disk device. Each line has up to three parts:

- Directories or partitions to be exported
- Options on that export
- Clients permitted to connect

Of the three components of an */etc/exports* entry, only the directory is mandatory. The directory path cannot contain symlinks, double dots, or single dots.

If I wanted to export my home directory as read-write to every host on the Internet, I could use an *exports* line containing only the path to my */home* folder:

```
/home/mwlucas
```

This perfectly valid (but perfectly foolish) entry contains no options and no host restrictions.

To export multiple directories that reside on the same partition, separate them with a single space.

```
/home/mwlucas /home/lasnyder
```

You can list any number of directories on one line, as long as they exist on the same partition.

NFS clients can mount only exactly the directory specified in */etc/exports*. If you export */home/mwlucas*, clients can attach only */home/mwlucas* to a mount point. They cannot mount, say, */home/mwlucas/bin* instead. If you would like to export an entire partition, you can do that, too. If you want to let clients mount any directories beneath that mount point, specify the mount point and the `-alldirs` option. You cannot use `-alldirs` with a subdirectory; it must be the actual mount point. This next entry lets anyone mount any directory in */home*:

```
/home -alldirs
```

To export multiple partitions, or directories from multiple partitions, specify them on separate lines.

```
/home -alldirs
/var/log
```

Any time you change */etc/exports*, you must signal `mountd` to reread its configuration. You can do this by passing the `reload` argument to the `mountd` startup script:

```
# /etc/rc.d/mountd reload
```

While these simple mounts give you an idea of how NFS works, they're very insecure. To make an intelligent export, you need a few options and an access list. Let's take a look at some of NFS's more commonly used options.

Read-Only Mounts

You might want to share files without worrying about whether your underlings will delete, modify, or otherwise undo your hard work. You can share files as read-only by using the `-ro` option. Here, I offer my home directory to all the computers in the world, but as a read-only share:

```
/home/mwlucas -ro
```

This is slightly more intelligent than offering my NFS exports to the entire world read-write, but only slightly.

NFS and Users

You already know that file ownership and permissions are tied to UID numbers. Unlike many other file-sharing protocols, NFS also uses UIDs to identify file ownership. For example, on my test server, my account `mwlucas` uses the UID 1000; on my client, my `mwlucas` account also uses the UID 1000. This simplifies my life, as I don't need to worry too much about file ownership; files owned by `mwlucas` on the server are owned by `mwlucas` on the client.

On a small network with only a few users and machines,[2] you can probably keep UID numbers synchronized without a problem by assigning the same UID to the same user on all of your systems. But on a large network, with more than one user and where users have root on their own machines, file ownership can quickly become a serious problem. The best way around this is to maintain a central repository of authorized users via LDAP or Kerberos.

Regardless of how you manage your users, NFS handles the root account differently. An NFS server cannot trust root on client machines to execute commands or write files as root on the server; if that were the case, a breach

2. How many users do I mean by "a few?" When synchronizing UIDs across all of your systems begins to really, *really* annoy you, you no longer have a few users.

on one NFS client would mean a breach on the NFS server. By default, requests from root on the client are mapped to UID and GID 32767 (also known as `nobody`).

If you want to map root to a specific user rather than the generic UID `nobody`, use the `-maproot` option and specify either a username or UID. Here, we map incoming requests from root on the client to the user `nfsroot` on the server:

```
/home/mwlucas -maproot=nfsroot
```

You can give the mapped root user a list of groups that the remote root account can access by specifying them after the username, separated by colons. Here, we give the client's root user access to the server as the user `nfsroot` and the groups `customers` and `webmasters`:

```
/home/mwlucas -maproot=nfsroot:customers:webmasters
```

If you want to explicitly remove the mapped root user from all groups, put a colon after the username or UID, as in this example:

```
/home/mwlucas -maproot=nfsroot:
```

Suppose you want all the NFS clients, regardless of username on the client system, to use a single user ID on the NFS server. The `-mapall` option allows you to do this. This option uses the same format as the `-maproot` option. Here, we map all NFS users to the username `nfsuser` on the server:

```
/home/mwlucas -mapall=nfsuser
```

Correct control of user access will help protect your NFS server.

Permitted Clients

By default, every host can access your NFS server. For many reasons, that's not a great idea. You can restrict the clients permitted to access your NFS server by listing their IP addresses at the end of the export entry.

```
/home/mwlucas 192.0.2.1
```

You can also specify clients by their hostname, but if the server has a DNS failure, it won't allow any clients access.

```
/home/mwlucas treble.blackhelicopters.org
```

To permit access to an entire network, use the `-network` and `-mask` options. The next example permits access to the addresses 192.0.2.0 through 192.0.2.15, using a subnet mask. (If you're not familiar with subnet masks, read Chapter 11.)

```
/home/mwlucas -network=192.0.2.0 -mask=255.255.255.240
```

When setting up your NFS server, I recommend you grant access to only the hosts who need it.

Multiple Exports for One Partition

You can have only one line for each combination of partition and permitted clients. If */home* is a single partition, you can't have an exports file that looks like this:

```
/home/mwlucas -maproot=nfsroot: 192.0.2.1
/home/pkdick 192.0.2.1
```

If two directories are located on the same partition, NFS will not allow you to export them to the same host using different permissions. You can, however, export directories on one partition to different hosts with different permissions, as shown here:

```
/home/mwlucas -maproot=nfsroot: 192.0.2.1
/home/pkdick 192.0.2.2
```

You can export directories on a partition to different hosts with different permissions.

```
/home/mwlucas -maproot=nfsroot: 192.0.2.1
/home/mwlucas -maproot=root 192.0.2.2
```

Only by combining IP restrictions and controlling user permissions can you can effectively control NFS server access.

NFS Clients

OpenBSD's NFS client doesn't need any daemons or configuration. Just `mount` the remote filesystem. Here, I mount my home directory from my server `treble` on */mnt*:

```
# mount treble:/home/mwlucas /mnt
```

When mounting remote filesystems over NFS, enter the hostname or IP address, a colon, and the directory. Because I have the same UID on both the client and server, I can access, alter, remove, and add files in */mnt* exactly as if I were dealing with files on a local filesystem.

Verify your mount with `df(1)` or `mount(8)`.

```
$ df -h
Filesystem             Size    Used   Avail Capacity  Mounted on
/dev/sd0a             1005M    266M    689M    28%    /
...
treble:/home/mwlucas  26.9G   21.5M   25.5G     0%    /mnt
```

The NFS-mounted directory shows up like any other mount point.

To mount an NFS share automatically at boot, or just record it for future convenience, you may use an */etc/fstab* entry. If your system might not have DNS available to it at boot time, use an IP address for the NFS server. The following example specifies two *fstab* entries: one using a hostname and one using an IP address:

```
treble:/home/mwlucas /mnt nfs,noauto rw 0 0
192.0.2.88:/usr/ports /usr/ports nfs,noauto ro 0 0
```

Give all NFS partitions dump and `fsck` numbers of `0`. Do not run `fsck` or `dump` on an NFS mount, as those programs require raw disk access that NFS doesn't provide.

Use any other mount options you like. The OpenBSD folks recommend using `noexec`, `nodev`, and `nosuid` "when applicable." I recommend `noauto` on NFS partitions that aren't required for normal server operation, so that an unavailable NFS server does not hang your machine's boot process.

NFS performance depends a great deal on your hardware, your local network, the clients and servers involved, the phase of the moon, and any number of other factors. If you're not happy with your NFS performance, read `mount_nfs(8)` and experiment with using TCP or UDP, the read and write sizes, and perhaps the timeout. If you need a complicated NFS environment, you should definitely invest some time in learning more about NFS.

Software RAID

The Redundant Array of Independent Disks (RAID) technology has become the standard way of mirroring hard drives within a machine or combining multiple hard drives to form one giant partition. In many types of RAID arrays, if one disk fails, the system can continue to run without data loss until you replace the failed disk or a second disk fails.

You can get RAID from the hardware or have the operating system perform the RAID operations. Hardware RAID controllers seem nice, but are in reality just decent disk controllers that run special software. Using the `softraid(4)` driver, OpenBSD can do the same thing, letting you build RAID arrays out of plain disks. You can do just about everything you can with a hardware RAID controller with a bunch of disks and OpenBSD's RAID management program `bioctl(8)` and the `softraid(4)` software RAID driver.

NOTE *In addition to managing software RAID, OpenBSD's `bioctl(8)` can manage most sorts of hardware RAID controllers. If you're planning to use hardware RAID, reading the `bioctl` manual is definitely worth your time.*

RAID Types

OpenBSD supports the following RAID configurations:

RAID-0, or *striping*
: This type is not redundant. It requires at least two disks of the same size, and data is shared between the disks to increase partition size and throughput. You can use RAID-0 to combine five 4TB disks into a 20TB virtual disk, but be warned: If one hard drive in the array fails, you'll lose all your data. RAID-0 is useful when you need a really big filesystem, but it's more vulnerable than a single disk because it provides multiple points of failure (or as one of my quasi-literary, quasi-humorous friends once said, "RAID-0 gives a whole new meaning to the phrase one disk to rule them all"). The size of a RAID-0 array is the size of all the hard drives combined.

RAID-1, or *mirroring*
: With this type, the contents of one disk are duplicated on another. Mirroring requires at least two disks of the same size, and the size of a RAID-1 array is equal to the size of the smallest drive in the array. I use mirroring to protect all vital data, as it gives even a cheap desktop-chassis server some measure of data protection. OpenBSD's software RAID fully supports this level.

RAID-4, or *striping data across disks, with a dedicated parity disk*
: This type requires at least three disks of the same size. Parity data lets a RAID array recover data on missing disks, and RAID-4 stores that parity data on a specific disk. This means that you can lose any one of the disks without losing data. As I write this, `bioctl`'s RAID-4 support is experimental. Hopefully this support will be complete before the book reaches you, but if not, you'll need to use a hardware RAID card to get RAID-4.

RAID-5, or *striping with parity shared across all drives*
: This is the current industry standard for redundancy. Parity data provides data redundancy—the loss of a single drive doesn't destroy any data. It requires at least three disks of the same size. Unlike RAID-4, RAID-5 shares the parity data across all the drives simultaneously. While throughput isn't as good as that of RAID-0, a RAID-5 array can simultaneously serve multiple I/O requests. The size of your RAID-5 array is the combined size of all but one of your hard drives. If you have five 4TB drives, the array will be 16TB ((5 – 1) × 4TB). Like RAID-4, RAID-5 support in `bioctl` is incomplete and experimental. I hope it will be complete before you read this, but if not, you'll need to use a hardware RAID card for RAID-5.

According to the RAID standards, each of these levels requires disks of the same size. That said, OpenBSD's `softraid` uses partitions rather than disks. You can use disks of different sizes, but your RAID array will use only

an amount of space on each disk equal to the smallest drive. If you want to mirror a 1TB drive and a 2TB drive, your mirror will offer only 1TB of space. The excess space on the larger drive is wasted.[3]

In addition to the standard RAID methods, `softraid` also allows you to encrypt your data across all disks in a RAID array (as described in "Encrypted Disk Partitions" on page 166). It also lets you *concatenate* disks. Concatenated disks are just run together to create one large virtual disk. You could concatenate two 500GB disks and a 1TB disk to create a single 2TB partition. These disks don't need to be the same size, but as with RAID-0, they are vulnerable. Damage to any one disk will completely wreck the virtual disk and lose all data. As the process for creating a concatenated disk closely resembles that of creating a RAID-0 disk, we'll cover it in "Creating softraid Devices" on page 163.

Preparing Disks for softraid

The `softraid` software RAID device builds its virtual disks out of disklabel partitions. To use a disk in a `softraid` array, prepare it just as you would a disk for a regular filesystem.

On i386 and amd64, disks underlying a `softraid` device need an MBR partition. To mark a whole disk with a single MBR partition, run `fdisk -i` on the disk.

Suppose you have five disks to use in a RAID array: `sd2`, `sd3`, `sd4`, `sd5`, and `sd6`. You'll need to prepare each of them as follows:

```
# fdisk -i sd2
Do you wish to write new MBR and partition table? [n] y
Writing MBR at offset 0.
```

Repeat this for every disk in your array.

Once you've added an MBR to all your disks, you'll need to put a disklabel partition on each disk. I tend to use partition letter *p* (the last available partition letter) for `softraid` devices. Here's how to set up a disk for `softraid`:

```
# disklabel -E sd2
Label editor (enter '?' for help at any prompt)
❶ > a
❷ partition: [a] p
offset: [64]
size: [104856191]
❸ FS type: [4.2BSD] RAID
❹ > q
❺ Write new label?: [y] y
```

3. You could add a non-RAID partition in the unused space on the larger drive, but that would do terrible things to your system's performance. Just buy more hard drives, you cheapskate.

First, we add a partition with `a` ❶ and assign it partition letter `p` ❷. Instead of our usual filesystem type of 4.2BSD, we assign a filesystem type of `RAID` ❸. Then we quit ❹ and let `disklabel` write the changes to the disklabel partition ❺.

If you have multiple identical disks, you can use `disklabel` to save this disk's configuration, as follows:

```
# disklabel sd2 > disklabel.sd2.raid
```

This saves the label on disk `sd2` to the file *disklabel.sd2.raid*. You can make `disklabel(8)` copy this partitioning to other disks, and `disklabel` will assign each disk a unique DUID as it copies. This saves you from needing to walk through the interactive editor for each disk. Let's apply this disklabel to each partition:

```
# disklabel -R sd3 disklabel.sd2.raid
# disklabel -R sd4 disklabel.sd2.raid
# disklabel -R sd5 disklabel.sd2.raid
# disklabel -R sd6 disklabel.sd2.raid
```

Disks `sd2` through `sd6` are now ready for assimilation into `softraid`.

Creating softraid Devices

Use `bioctl(8)` to drag disks into a software RAID. You'll need the disk partitions you want to include in the RAID. OpenBSD software RAID arrays are named `softraid`, followed by a number. Use the `-c` argument to give a RAID type, and `-l` to give the partitions, and end with the name of the `softraid` you're creating.

```
# bioctl -c raidlevel -l partition1,partition2... softraidX
```

We have five disk partitions—`sd2p`, `sd3p`, `sd4p`, `sd5p`, and `sd6p`—to add to a `softraid` device. To build a RAID-5 device out of these partitions, run this command:

```
# bioctl -c 5 -l sd2p,sd3p,sd4p,sd5p,sd6p softraid0
softraid0: SR ❶ RAID 5 volume attached as ❷ sd7
```

The response indicates that we've successfully created a RAID-5 device ❶, and it's available as device `/dev/sd7` ❷. On a blank RAID disk, which you need to prepare just as you would any other new disk, run `fdisk -i sd7` and `disklabel` to create MBR and OpenBSD partitions, use `newfs` to create a filesystem on the new partitions, and you're ready to go. (See the instructions for adding a new disk in Chapter 8 for details.)

You could have made this a RAID-0, RAID-1, or RAID-4 device by choosing a different `-c` option. The tricky one is a concatenated `softraid`. To dump all the disks together into a single concatenated virtual partition, use `-c c`.

```
# bioctl -c c -l sd2p,sd3p,sd4p,sd5p,sd6p softraid0
softraid0: SR CONCAT volume attached as sd7
```

softraid Status

To check the health of each device in a RAID array, give `bioctl` the device name of the `softraid` device.

```
# bioctl softraid0
Volume      Status               Size Device
softraid0 0 Online       214744170496 sd7     RAID5
          0 Online        53686099456 0:0.0   noencl <sd2p>
          1 Online        53686099456 0:1.0   noencl <sd3p>
          2 Online        53686099456 0:2.0   noencl <sd4p>
          3 Online        53686099456 0:3.0   noencl <sd5p>
          4 Online        53686099456 0:4.0   noencl <sd6p>
```

We see that the five drives are in use, all assembled into a RAID-5 virtual drive. Everything here is healthy. Anything that doesn't look roughly like this indicates a problem.

Identifying Failed softraid Volumes

If you have a RAID-1, RAID-4, or RAID-5 `softraid` volume, you can lose a drive and not lose your data. `bioctl` tells you if a drive fails. Here, one of the drives in my `softraid` volume has failed:

```
# bioctl softraid0
Volume      Status               Size Device
softraid0 0 Degraded     214744170496 sd7     RAID5
          0 Online        53686099456 0:0.0   noencl <sd2p>
          1 Offline                0 0:1.0    noencl <>
          2 Online        53686099456 0:2.0   noencl <sd3p>
          3 Online        53686099456 0:3.0   noencl <sd4p>
          4 Online        53686099456 0:4.0   noencl <sd6p>
```

Looking closely at this, I can see that drives `sd2`, `sd3`, `sd4`, and `sd6` are still available and in use. All my data should still be intact, but I need to replace `sd5` before another disk fails.

Rebuilding Failed softraid Volumes

As of this writing, you cannot rebuild a failed `softraid` RAID-4 or RAID-5 device. You must back up your data, replace the failed drive, delete the `softraid` device, re-create the filesystem, and restore from backup. You can, however, rebuild a RAID-1 device.

Let's look at replacing a disk in a RAID-1 device. Here's what a healthy, three-disk `softraid` mirror might look like:

```
# bioctl softraid0
Volume      Status               Size Device
softraid0 0 Online        53686099456 sd5❶  RAID1
          0 Online        53686099456 0:0.0   noencl <sd2p>❷
          1 Online        53686099456 0:1.0   noencl <sd3p>
          2 Online        53686099456 0:2.0   noencl <sd4p>
```

Note that this RAID device has device node `sd5` ❶ and includes the partitions *sd2p*, *sd3p*, and *sd4p* ❷.

We replace two disks and reboot this machine. Suddenly, the *softraid* device looks very different.

```
# bioctl softraid0
Volume      Status               Size Device
softraid0 0 Degraded      53686099456 sd5     RAID1
          0 Offline               0 0:0.0   noencl <>
          1 Offline               0 0:1.0   noencl <>
          2 Online        53686099456 0:2.0   noencl <sd2p>
```

Partitions *sd3p* and *sd4p* are missing. That's because the underlying disks have been replaced.[4] Prepare the replacement disks for software RAID, as discussed in "Preparing Disks for softraid" on page 162. Then run `bioctl`, using the `-R` flag to specify the disk to replace in the `softraid` device.

```
# bioctl -R /dev/sd3p sd5
softraid0: rebuild of sd5 started on sd3p
```

If you check the status of the device using `bioctl`, you'll see the disk status now says "Rebuilding."

If you have a mirror with more than two disks, you must rebuild each disk separately. Rebuild the first disk, and then rebuild the second disk.

Deleting softraid Devices

To remove a `softraid` device from your system, pass `bioctl` the `-d` flag and the device name for the `softraid` device. Here's how to remove the RAID-5 device we just created:

```
# bioctl -d sd7
```

WARNING *Once you delete the RAID device, you can't get it back unless you re-create it and restore your data from backup.*

4. If you need to force an error on a hard disk, removing the disk from the machine will certainly do it.

Reusing softraid Disks

softraid writes metadata at the beginning of the disks it uses. You need to overwrite this metadata before you can use the disks in another softraid device. Overwrite the first megabyte or so of the disk with dd(1).

```
# dd if=/dev/zero of=/dev/sd2c bs=1k count=1024
1024+0 records in
1024+0 records out
1048576 bytes transferred in 0.594 secs (1765074 bytes/sec)
```

This erases the MBR partitions, any initial disklabels, and any filesystem information on the disk. You can now reuse these disks in softraid devices as normal disks.

Booting from a softraid Device

The softraid feature is still in development. Eventually, you'll be able to use the installer to build a software RAID device, install OpenBSD on that device, and run a full RAID configuration out of the box. But as I write this, you'll need to jump through some hoops to make that happen. Rather than document a specific procedure that will change as OpenBSD completes softraid development, I'm going to tell you to search the Internet and the *misc@OpenBSD.org* archives for the most recent instructions.

Encrypted Disk Partitions

Sometimes I can see the future. When someone says, "I've encrypted my hard drive!" I have a psychic vision of them saying "I've lost all my data!" While encrypting a hard drive partition is warranted in some cases, most of the time, it's just pretentious. In this section, I will do you the courtesy of assuming that you understand when you truly need disk encryption if you will do me the courtesy of not complaining to me when you lose your data.[5]

Creating Encrypted Partitions

OpenBSD includes disk encryption as a bioctl(8) option—specifically, like a RAID discipline. Where disk activity would normally be passed through a RAID discipline, here they pass through an encryption discipline. The encrypted disk even shows up as a softraid device. Much like the support for RAID-5, support for encrypted filesystems is experimental. Although it *should* work, don't be shocked if some features are not yet included or if it eats your entire disk. Keep good backups. Reread the previous paragraph. And again—*please* don't complain to me when it doesn't work.

5. Not that I can help you—all I can do is say "I told you so." On a related note: You can get tired of anything, no matter how pleasant, if you have to do it often enough.

Under OpenBSD, an encrypted volume can include only a single partition. Use the RAID type `C` to specify an encrypted volume. Here's, how to create an encrypted volume on the *sd4p* partition:

```
# bioctl -c C -l sd4p softraid0
❶ New passphrase:
Re-type passphrase:
softraid0: SR CRYPTO volume attached as sd5
```

When prompted ❶, enter a passphrase twice. A good passphrase is several words long, and includes a mix of characters, symbols, numbers, punctuation, and whitespace. The passphrase is the secret code used to encrypt and decrypt data, so the longer and more varied it is, the better. Remember this passphrase; you must enter it again to recover your data. Once you've entered your passphrase twice, `bioctl` creates the encrypted disk device. In this case, it has created encrypted disk `softraid0` as `disk sd5`.

Using Encrypted Partitions

Do not mount this new disk yet! Instead, use `fdisk` to check our new, encrypted partition.

```
# fdisk sd5
Disk: sd5       geometry: 6526/255/63 [104855663 Sectors]
Offset: 0       Signature: 0x8BF9
            Starting         Ending         LBA Info:
 #: id      C   H   S -      C   H   S [       start:        size ]
-------------------------------------------------------------------------------
 0: D9 230285  63  36 - 134263  55  58 [  3699532529:  2752373385 ] <Unknown ID>
 1: 8C  73068 221  44 - 176434  56  49 [  1173851386:  1660564401 ] <Unknown ID>
 2: C9 218148  78  47 - 141866 243  13 [  3504552580:  3069507328 ] <Unknown ID>
 3: AC 125252   6   1 - 245307  77  22 [  2012173758:  1928688070 ] <Unknown ID>
```

The underlying disk is blank, and our `fdisk` output looks like garbage, but this disk is now an encrypted volume.

Now that the encrypted disk exists, create an MBR partition and add disklabel partitions, just as when you add any other disk. Then you can mount your encrypted device partition using the device node—again, just as with any other disk.

To unmount the decrypted partition, destroy the `softraid` device by passing `bioctl` the `-d` argument.

```
# bioctl -d sd5
```

To anyone who doesn't have the passphrase, this partition now looks like random garbage.

Automatic Decryption

If you have an encrypted partition, presumably you don't want OpenBSD to automatically decrypt and mount it when the system boots. (The whole point of an encrypted partition is that only a person who has the passphrase can access the encrypted data.) Still, I'm not one to tell you not to shoot yourself in the foot, so if you must automatically decrypt the partition, you can do so.

First, create a file containing your passphrase. Give ownership of this file to root and set the permissions to `600` (read-write by owner; no access by other users), and then give this file to `bioctl(8)` with the `-p` flag. In this example, the encrypted disk is created as */dev/sd5* and there is a partition on */dev/sd5a*. I've stored my passphrase in the file */etc/passphrase*, so I could run something like this:

```
# bioctl -c C -l sd4p -p /etc/passphrase softraid0
# mount /dev/sd5a /home/mwlucas
```

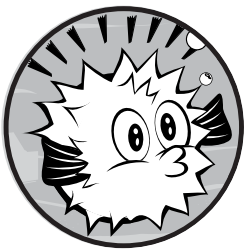
Adding this to */etc/rc.securelevel* will mount this encrypted partition at boot.

You should now have a good idea of how to manage OpenBSD disks and filesystems. Next, we'll look at some of OpenBSD's special security features.

10

SECURING YOUR SYSTEM

Hackers at the gates?
Puffy the Barbarian
defends against fiends.



Securing your system means ensuring that your computer's resources are used only by authorized people and for authorized purposes. Even if a system has no important data, it still has valuable CPU time, memory, storage, and bandwidth. People who think that their systems are too unimportant for anyone to bother breaking into risk finding their equipment hosting pornography or relaying attacks against industrial or military sites. If you're like me, you would rather not discover that your computers took down a government agency by having law enforcement agents kick in your door.

Taking over large numbers of remote computers gets easier all the time. Every year, more and more point-and-click toolkits for penetrating servers crop up. When one bright attacker posts an exploit, anyone can use it. Breaking into computers is big business, and if your computer is left unprotected, it *will* be penetrated. The only question is how.

Generally speaking, intruders don't break into operating systems; they break into server programs running on the operating system. Even the most paranoiac, secure-by-default operating system cannot protect poorly written programs from themselves. OpenBSD features like W^X and address space layout randomization do a lot to protect the operating system from the side effects of buggy programs, but programs themselves still crash and burn. OpenBSD has undergone extensive auditing and testing to eliminate the most common security flaws, but there's no guarantee that every security flaw has been eradicated. New features appear constantly, and can interact with older functions (and each other) in unexpected ways. For more details on the OpenBSD-specific features, check the papers and presentations collection at *http://www.OpenBSD.org/papers/*.

No single tool can protect your server against all threats, and no single tool is applicable to all environments. Learning about OpenBSD's security features helps you to understand not only what the tools do, but when they should be used and when they won't help your particular situation. The best place to start is by understanding the threat.

Who Is the Enemy?

Books dedicated to security break attackers down into smaller, more specific groups and include various edge cases, but that's not what you're here for. I lump potential attackers into four groups: script kiddies, botnets, disaffected users, and skilled attackers. These categories are easily understood and include 99 percent of all the attackers you're likely to encounter.

Script Kiddies

The most common type of attackers, script kiddies, are not sysadmins. They are amateurs who download attack scripts and go looking for poorly defended, vulnerable systems.

Script kiddies are easy to defend against: Keep your software up-to-date and follow good computing practices. Like locusts, script kiddies are easy to squash, but there are just so darned *many* of the little buggers!

Botnets

Botnets are composed of machines compromised by worms or viruses and are controlled from a central point. The botnet's controllers might use the victim machines to search for more vulnerable hosts, to send spam, or to break into secure sites. Most botnets are composed of Windows or Linux machines, but there's no reason why such a worm couldn't target OpenBSD. The virus author would need to work hard, but it's conceivable—if he finds a suitable security flaw.

Fortunately, botnet defense is much like script kiddie defense. You shouldn't have much to worry about if you keep your software patched, configure your server software securely, and follow good computing practices.

Disaffected Users

Security pundits commonly claim that a system's legitimate users cause the majority of security problems.[1] Legitimate users are most likely to know where your security gaps are, to feel that the system rules don't apply to them, and to have the necessary access and time to experiment with breaking your security. If you tell an employee that company policy forbids him access to a computer resource, and the employee feels that he *should* have access to it, he is likely to search for a way around the restriction. You can patch all of your servers and protect them with an outright hostile firewall, but if someone has physical access and knows the root password, your protections are useless.

Deal with this problem on two levels. The first is technical: Keep your servers patched and up-to-date. The second is human: Don't leave projects half finished or half documented. That unsecured modem you installed for emergency incoming access until the VPN is solid? Get rid of it, or put a password on it. Ditto for that telnet server running on a nonstandard port.

Security by obscurity is feeble at best. When a privileged user leaves the company, immediately disable his account, change all administrative passwords, inform employees of the person's departure, and remind them not to share confidential information with that person. Implement a computer security policy with real penalties for violations. If you have a Human Resources department, get the staff members to agree to the policy and insist they enforce it.

What's the best way to protect yourself against the disaffected user? Don't be lazy.

Skilled Attackers

As the most dangerous group, skilled attackers are competent system administrators, security researchers, penetration specialists, and criminals who want access to specific resources. Taking over computers is a lucrative business these days. Sending junk email or launching distributed denial-of-service attacks can bring in large sums of money. These intruders don't care who they attack, as long as they secure the computing resources they need.

If your company has valuable secrets, however, you might attract an entirely different type of intruder: someone who wants access to your network in particular. If your employer creates anything—from software to cast-iron tulips for front-wheel-drive vehicles—there's likely a market for illicit copies of your product. Someone will find it worthwhile to probe every port on every IP address you expose to the Internet. It might take a long time, but that's okay. Your data has a price tag, and the scan is cheap. This is often called the advanced persistent threat, or APT.

---

1. I've seen too many botnet or script kiddie intrusions go undetected for months to be comfortable blaming legitimate users for the majority of security problems. I would agree that "insider intrusions" are the most commonly identified intrusions, but frequently, that's because the guilty user can't keep his mouth shut.